LAPORAN PRAKTIKUM STRUKTUR DATA

PEKAN 3

Oleh: Afif Naufal Zahran

NIM: 2511533009

DOSEN PENGAMPU: DR.WAHYUDI, S.T, M.T

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

Kata Pengantar

Pedoman ini disusun sebagai rujukan resmi bagi mahasiswa Departemen Informatika dalam penyusunan laporan praktikum pada mata kuliah *Struktur Data*. Dokumen ini tidak hanya memberikan gambaran umum mengenai format penulisan, tetapi juga menguraikan secara rinci sistematika laporan, tata cara penyajian isi, serta contoh penulisan kode program yang dilengkapi dengan referensi ilmiah. Melalui panduan ini, mahasiswa diharapkan mampu menyusun laporan yang tidak sekadar memenuhi aspek administratif, tetapi juga mencerminkan ketelitian, keteraturan, dan penerapan kaidah penulisan akademik pada tingkat dasar. Dengan demikian, laporan praktikum yang dihasilkan dapat berfungsi sebagai media pembelajaran, dokumentasi kegiatan, sekaligus sarana untuk melatih keterampilan menulis ilmiah yang akan bermanfaat dalam jenjang studi selanjutnya.

Padang, 2025

Tim Penyusun

DAFTAR ISI

BAB I .....

1.1 Latar Belakang.....

1.2 Tujuan .....

1.3 Manfaat .....

BAB II .....

2,1 Teori.....

2,2 Program.....

BAB III.....

3,1 Kesimpulan.....

3,2 Saran .....

BAB I

PENDAHULUAN

1.1 Latar Belakang

ADT (Abstraction Data Tpe) merupakan konsep yang cukup penting apalagi dalam pemrograman, terutama dalam pembuatan sebuah program. Memahami konsep ADT dapat membuat program jauh lebih mudah dan lebih mudah dalam pemeliharaa(*Maintenance*).

Stack merupakan struktur data linier yang mengikuti prinsip Last In First Out (LIFO), di mana elemen terakhir yang dimasukkan adalah yang pertama dikeluarkan

1.2 Tujuan

Tujuan dari laporan praktikum ini agar pembaca dan penulis dapat mendalami dan memahami tentang ADT dan Stack serta pembuatan ADT dan Stack di java.

1.3 Manfaat

Manfaat dari praktikum tentang operator agar pembaca dan penulis dapat mendalami dan memahami tentang ADT dan Stack serta pembuatan ADT dan Stack di java.

BAB II

PEMBAHASAN

2,1 Teori

ADT(Abstract Data Type) merupakan model teoretis untuk struktur data yang menentukan perilakuk data (operasi yangbisa dilakukan) tanpa menentukan cara implementasinya.

Stack merupakan struktur data linier yang mengikuti prinsip Last In First Out (LIFO), di mana elemen terakhir yang dimasukkan adalah yang pertama dikeluarkan

2,2 Program

Terdapat 5 program yakni, StackPostfix, stackArray, stackArayDriver, SiswaStack, dan NilaiMaksimum

a. NilaiMaksimum_2511533009

```java
package pekan3_2511533009;

import java.util.Stack;

public class NilaiMaksimum_2511533009 {
    public static int max_3009(Stack<Integer> s_3009){
        Stack<Integer> backup_3009 = new Stack<Integer>();
        int maxValue_3009 = s_3009.pop();
        backup_3009.push(maxValue_3009);
        while (!s_3009.isEmpty()){
            int next_3009 = s_3009.pop();
            backup_3009.push(next_3009);
            maxValue_3009 = Math.max(maxValue_3009,next_3009);
        }
        while (!backup_3009.isEmpty()) {
            s_3009.push(backup_3009.pop());
        }
        return maxValue_3009;
    }
    public static void main(String[] args){
        Stack<Integer> s_3009 = new Stack<Integer>();
        s_3009.push(70);
        s_3009.push(12);
        s_3009.push(20);
        System.out.println("isi stack " + s_3009);
        System.out.println("Stack Teratas " + s_3009.peek());
        System.out.println("Nilai maksimum " + max_3009(s_3009));
    }
}
```

Program berikut merupakan contoh penggunaan Stack pada java, dimana kita dapat melakukan Push (menambahkan) dan melihat data teratas(peek)

b. SiswaStack_2511533009

```
package pekan3_2511533009;

import java.util.ArrayList;

class Siswa_2511533009{
    String nama_3009;
    int nim_3009;
    public Siswa_2511533009(String nama_3009,int nim_3009){
        this.nama_3009 =nama_3009;
        this.nim_3009 =nim_3009;
    }

    @Override
    public String toString(){
        return "Nim: " + nim_3009 + ", Nama: " + nama_3009;
    }
}
public class SiswaStack_2511533009 {
    private ArrayList<Siswa_2511533009> stack_3009;

    public SiswaStack_2511533009(){
        stack_3009 = new ArrayList<>();
    }

    public void push_3009(Siswa_2511533009 mhs_3009){
        stack_3009.add(mhs_3009);
    }

    public Siswa_2511533009 pop_3009(){
        if (!isEmpty_3009()){
            return stack_3009.remove(stack_3009.size() - 1);
        }
        return null;
    }

    public Siswa_2511533009 peek_3009(){
        if (!isEmpty_3009()) {

            return stack_3009.get(stack_3009.size()-1);
        }
        return null;
    }

    public boolean isEmpty_3009(){
        return stack_3009.isEmpty();
    }

    public void tampilkanSiswa_3009(){
        for (int i = stack_3009.size() - 1; i>=0;i--){
```

```
            System.out.println(stack_3009.get(i));
        }
    }
    public static void main(String[] args){
        SiswaStack_2511533009 studentStack_3009=  new SiswaStack_2511533009();

        Siswa_2511533009 mhs1_3009 = new Siswa_2511533009("Ali",1);
        Siswa_2511533009 mhs2_3009 = new Siswa_2511533009("Boby",2);
        Siswa_2511533009 mhs3_3009 = new Siswa_2511533009("Charles",3);
        studentStack_3009.push_3009(mhs1_3009);
        studentStack_3009.push_3009(mhs2_3009);
        studentStack_3009.push_3009(mhs3_3009);

        System.out.println("Siswa di dalam stack:");
        studentStack_3009.tampilkanSiswa_3009();

        System.out.println("siswa teratas " + studentStack_3009.peek_3009());
        System.out.println("mengeluarkan siswa teratas dari stack: " +
studentStack_3009.pop_3009());
        System.out.println("daftar siswa setelah di pop :");
        studentStack_3009.tampilkanSiswa_3009();
    }
}
```

Program berikut merupakan contoh Kolaborasi ADT dan Stack pada 1 file yang sama namun berbeda kelas, pada kelas SiswaStack_2511533009 merupakan class untuk handle Stack, sedangkan Siswa_2511533009 merupakan ADT.

c. stackArray_2511533009

```
package pekan3_2511533009;

public class stackArray_2511533009 {
    static final int MAX_3009 = 1000;
    int top_3009;
    int a_3009[] = new int[MAX_3009];
    boolean isEmpty_3009()
    {
        return (top_3009<0);
    }
    stackArray_2511533009()
    {
        top_3009= -1;
    }
    boolean push_3009(int x)
    {
        if (top_3009 >= (MAX_3009 - 1)){
            System.out.println("Stack overflow");
            return false;
        }
        else {
            a_3009[++top_3009] = x;
            System.out.println(x + " dimasukka dalam stack");
            return true;
```

```
        }
    }
    int pop_3009()
    {
        if (top_3009 <0) {
            System.out.println("Stack underflow");
            return 0;
        }
        else {
            int x_3009 = a_3009[top_3009--];
            return x_3009;
        }
    }
    int peek_3009() {
        if (top_3009 < 0) {
            System.out.println("Stack Underflow");
            return 0;
        }
        else {
            int x_3009 = a_3009[top_3009];
            return x_3009;
        }
    }
    void print_3009(){
        for (int i = top_3009;i >-1;i--){
            System.out.print(" " + a_3009[i]);
        }
    }

}
```

Sedangkan pada program berikut merupakan contoh cara kerja Stack menggunakan Array normal (tanpa java.util.Stack). Dimana pada program berikut bisa melakukan peek, pop, push, dan print serta dapat melakukan pengecekan jika data stack kosong atau tidak.

d. stackArrayDriver_2511533009

```
package pekan3_2511533009;

public class stackArrayDriver_2511533009 {

    public static void main(String[] args) {
        stackArray_2511533009 s_3009 = new stackArray_2511533009();
        s_3009.push_3009(10);
        s_3009.push_3009(20);
        s_3009.push_3009(30);
        System.out.println(s_3009.pop_3009() + " dikeluarkan dari stack");
        System.out.println("Elemen teratas adalah :" + s_3009.peek_3009());
        System.out.print("Elemnt pada stack :");
        s_3009.print_3009();
    }
```

```
}
```

Sedangkan pada program berikut kita menggunakan stackArray yang telah kita buat pada program sebelumnya, dan melakukan input data.

e. StackPostfix_2511533009

```
package pekan3_2511533009;

import java.util.Scanner;
import java.util.Stack;

public class StackPostfix_2511533009 {
    public static int postfixEvaluate(String expression){
        Stack<Integer> s_3009 = new Stack<Integer>();
        Scanner input_3009 = new Scanner(expression);
        while (input_3009.hasNext()){
            if (input_3009.hasNextInt()) {          // an operand (integer)
                s_3009.push(input_3009.nextInt());
            } else {                                // an operator
                String operator_3009 = input_3009.next();
                int operand2_3009 = s_3009.pop();
                int operand1_3009 = s_3009.pop();
                if (operator_3009.equals("+")){
                    s_3009.push(operand1_3009+operand2_3009);
                } else if (operator_3009.equals("-")){
                    s_3009.push(operand1_3009-operand2_3009);
                } else if (operator_3009.equals("*")){
                    s_3009.push(operand1_3009*operand2_3009);
                } else{
                    s_3009.push(operand1_3009/operand2_3009);
                }
            }
        }
        input_3009.close();
        return s_3009.pop();
    }
    public static void main(String[] args ){
        System.out.println("hasil postfix= " + postfixEvaluate("5 2 4 * + 7 -
"));
    }
}
```

Pada program berikut merupakan contoh penggunaan Stack dimana kita menambahkan data pada stack dan melakukan kalkulasi sesuai dengan operator yang ada pada string yang kita buat (seperti pada parameter method postfixEvaluate)

BAB III

PENUTUPAN

3,1 Kesimpulan

ADT dan Stack pada bahasa pemrograman baik itu *Java* ataupun bahasa pemograman yang lain karena hal yang sangat fundamental pada bahasa pemrograman.

3,2 Saran

Laporan praktikum ini masih memiliki kekurangan. Karena itu, penulis sangat terbuka terhadap saran dan kritikan agar dapat meningkatkan kualiats laporan ini dan laporan-laporan selanjutnya.